

Computer 1

Loads program from from Disk into RAM.

Computer 2

Program in RAM and has an input device & an output device I/O as per the "But How Do It Know ?" book.

Computer 3

Program in ROM and has RAM, 2 input devices & one output device using memory mapped I/O.

Computer 4

‘Computer 4’ is based on ‘Computer 3’ plus a stack pointer. This resulted in the CLF command being discarded.

Computer 5

‘Computer 5’ is based on ‘Computer 4’ plus a 2nd output device & the re-instatement of CLF.

Computer 6

‘Computer 6’ is based on ‘Computer 5’ but with the 8 bit bus/registers updated to 16 bit.

A top level ‘User Panel’, giving a more user friendly view of the computer’s state, has been added.

User entry is now assisted with the use of 'wait' LEDs & 'enter' buttons:

- When a program requires input the WAIT_x LED is turned on, and the data from IN_x is read on the ENTER_x button being pressed.

Also, the computer has a BIOS located in ROM, with programs being loaded in RAM. At the very least the BIOS should have, at address 0x0000, a jump to the starting address of the program.

Computer 7

‘Computer 7’ is based on ‘Computer 6’ but with an updated Control Unit that has microcode stored in a ROM.

‘Computer 7 (Matrix)’ is based on ‘Computer 6’ but with an updated Control Unit that uses a ‘command activation matrix’ to set its outputs dependant on the command being performed.

Computer 8

‘Computer 8’ is based on ‘Computer 7’ but with 2 more general purpose registers added, as well as a frame pointer and a link register. This has allowed for the creation of a a basic compiler.

The following commands have been added to the instruction set:

LDX RA,C,RB	: Load RB from the address (RA+C)
STX RA,C,RB	: Store RB to the address (RA+C)
MOV RA,RB	: Copy the contents of RA to RB
MOVSP RB	: Copy the stack pointer to RB

Computer 9

‘Computer 9’ is based on ‘Computer 8’ but with the following Control Unit changes to allow for any future expansion of the number of instruction cycles:

- The Stepper has been removed and counters are used for the instruction fetch and instruction execute cycles.
- Added a look up table and an adder to determine the current microcode within an instruction.

From a software point of view the RAM will contain Code, Heap & Stack sections, so the memory map is now:

0x0000 - 0x7FFF	: ROM
0x8000 - 0xBFFF	: Code (RAM)
0xC000	: Heap start - grows upwards (RAM)
0xFFEF	: Stack start - grows downwards (RAM)
0xFFF0 - 0xFFFF	: IO Devices

The Compiler has been updated to use this memory map.

The BIOS has also been replaced with an operating system (TimmiOS).

Computer 10

‘Computer 10’ is based on ‘Computer 9’ but a maths unit has been added which performs 2’s-complement integer subtraction, multiplication and division. An FPU has also been added which performs 16 bit floating point addition, subtraction, multiplication, division and square roots.

The following commands have been added to the instruction set:

SUB RA,RB	: $RB = RA - RB$
MUL RA,RB	: $RB = RA * RB$
DIV RA,RB	: $RB = RA / RB$
FADD RA,RB	: $RB = RA + RB$
FSUB RA,RB	: $RB = RA - RB$
FMUL RA,RB	: $RB = RA * RB$
FDIV RA,RB	: $RB = RA / RB$
FSQRT RA,RB	: $RB = \sqrt{RA}$

Computer 11

‘Computer 11’ has the same hardware as ‘Computer 10’ but there have been changes to some of the instruction set formats:

LD RA,RB	→ LD RC,[RA]
LDX RA,C,RB	→ LDX RC,[RA,RB]
DATA RB,data	→ LDI RC,data
ST RA,RB	→ ST RC,[RA]
STX RA,C,RB	→ STX RC,[RA,RB]

Computer 12

‘Computer 12’ is based on ‘Computer 11’ but interrupt handling has been added, together with a CORDIC unit within the FPU, which adds Sin, Cos & Tan instructions to the instruction set.

The following assembly code instructions have been added:

STI data,[RC]	: Store data to the memory address in RC
SWAP RA,RB	: Swap the contents of registers RA & RB
DSI	: Disable interrupts
ENI	: Enable interrupts
RTI	: Return from interrupt
CLR RA	: RA = 0 (alias for: XOR RA,RA,RA)
NEG RA	: RA = -RA
FSIN RA,RC	: RC = SIN(RA)
FCOS RA,RC	: RC = COS(RA)
FTAN RA,RC	: RC = TAN(RA)

Also, the following instruction has been removed:

CLF : Clear flags

SAL-16A

‘SAL-16A’ (Sublime Applied Logic) is based on ‘Computer 12’ but a keyboard, display & graphics screen have been added.

Interrupt handling has been updated to allow the keyboard to generate interrupts, in addition to the ‘ENTER_1’ and ‘ENTER_2’ buttons.

The Instruction Set has had a number of updates, and the latest version is detailed in “Instruction Set.pdf”.

SAL-16F-B

‘SAL-16F-B’ is based on ‘SAL-16A’ but with Halt, Overflow and Error Trapping functionality added. Also, the DIV instruction now performs signed division, with UDIV being used for unsigned division.

The following assembly code instructions have been added:

JV addr	: Jump to addr if overflow
JNV addr	: Jump to addr if no overflow
HALT	: Halt the program
UDIV RA,RB,RC	: $RC = RA / RB$ (unsigned)

And the following instructions have been removed:

NEG RA	: $RA = -RA$
MOD RA,RB,RC	: $RC = RA \% RB$

A Linker has been added allowing several machine code files to be combined into a single file that can then be loaded into SAL-16’s memory.

There have been the following Compiler updates:

- ‘float’ and ‘char’ types are now supported
- Pointer and array handling has been added
- Libraries can now be imported

SAL-16F-C

‘SAL-16F-C’ is based on ‘SAL-16F-B’ but with circuitry added to allow floating point co-ordinates to be used for the Graphics Screen. This resulted in the Cache circuitry being removed because the FSETPX instruction replaced the SWAP instruction.

Some FPU/CLKE interaction circuitry was added to ensure that the FPU’s X input is present on the data bus during Divider/Square Root/CORDIC calculations.

The ALU was updated so that comparisons between the ‘A’ and ‘B’ inputs are now signed comparisons for integer comparisons. Also, the functionality for performing floating point comparisons has been added to the ALU.

The following assembly code instructions have been added:

FSETPX RA,RB,RC	: Set the pixel at position (RA, RB) to the colour in RC
FCMP RA,RB	: Calculate $(RA - RB)$ and set flags

And the following instruction has been removed:

SWAP RA,RB : Swap RA and RB

SAL-16F-D

‘SAL-16F-D’ is based on ‘SAL-16F-C’ but with functionality added to detect stack violations, i.e. if the stack pointer enters the heap or I/O region.

The following command has been added to the instruction set:

SETHP [RA] : Set heap end in the SP from the mem addr in RA

SAL-16F-E

‘SAL-16F-E’ has the same hardware as ‘SAL-16F-D’ but the following instruction set changes have been made:

LDI RC,data	→ MOV data,RC
LD RC,[RA]	→ MOV [RA],RC
LDX RC,[RA,RB]	→ MOV [RA,RB],RC
ST RA,[RC]	→ MOV RA,[RC]
STX RA,[RB,RC]	→ MOV RA,[RB,RC]
SETHP [RA]	→ ENDHP[RA]

Some simple code optimisation has also been added to the Compiler.

SAL-16I-x

‘SAL-16I-x’ is based on ‘SAL-16F-x’ but with the FPU and associated circuitry removed. Also added in some additional ‘interrupt’ circuitry so that programs will halt if they try using floating point instructions.

This allows for increased program speeds when no floating point operations are required.

SAL-16J-x

‘SAL-16J-x’ is based on ‘SAL-16F-x’ but using the ‘chips’ from the ‘SAL-16 Chips’ Java Library.

‘SAL-16J-x’ also includes a Sound Card that uses the ‘Beeper’ from the ‘SAL-16 Chips’ Java Library.

This allows for increased program speeds as opposed to using 'chips' built from Logisim Primitives.

SAL-16V-x

'SAL-16V-x' is a stripped down version of 'SAL-16I-x' that is used as the basis for the Verilog version of 'SAL-16'.